

DocRAG — Codebase Refactor Report

This document describes the structural refactor applied to the DocRAG backend. The goal was to bring the codebase into alignment with the [HackSoft Django Styleguide](#), a set of conventions for keeping Django projects maintainable as they grow: thin views, explicit service and selector layers, domain-scoped exceptions, and I/O-split serializers.

The refactor was applied across 70 files spanning three domains ([accounts](#), [documents](#), [qa](#)) and cross-cutting infrastructure. The test suite grew from 77 to 99 tests, all passing, at 84% overall coverage.

1. Cross-cutting: Exception Hierarchy

Before: errors were raised directly as DRF [ValidationError](#) or [Http404](#) inside views and serializers, mixing presentation concerns with domain logic.

After: a base [AppError](#) lives in [core/exceptions.py](#). Every domain defines its own subhierarchy from it, and a single global handler converts any [AppError](#) to a JSON response without any domain-specific knowledge.

```
# core/exceptions.py

class AppError(Exception):
    status_code: int = status.HTTP_400_BAD_REQUEST

    def __init__(
        self,
        message: str,
        status_code: int | None = None,
        details: dict[str, Any] | None = None,
    ) -> None:
        super().__init__(message)
        self.message = message
        self.details: dict[str, Any] = details or {}
        if status_code is not None:
            self.status_code = status_code

def custom_exception_handler(exc, context):
    if isinstance(exc, AppError):
        return Response(
            {"error": exc.message, "details": exc.details},
            status=exc.status_code,
        )
    return exception_handler(exc, context)
```

Each domain then defines its own hierarchy:

```
# accounts/exceptions.py

class AccountsServiceError(AppError):
    status_code = status.HTTP_400_BAD_REQUEST

class UserAlreadyExistsError(AccountsServiceError):
    status_code = status.HTTP_409_CONFLICT

    def __init__(self, field: str = "email") -> None:
        super().__init__(
            message="A user with this credential already exists.",
            details={field: "Already in use."},
        )

class UserNotFoundError(AccountsServiceError):
    status_code = status.HTTP_404_NOT_FOUND

    def __init__(self) -> None:
        super().__init__(message="User not found.")
```

`documents` and `qa` follow the same pattern with `DocumentServiceError` and `QAServiceError` respectively. Adding a new error type to any domain requires zero changes to `core/exceptions.py`.

2. Cross-cutting: Settings Split

Before: a single `core/settings.py` loaded all configuration for every environment.

After: a `core/settings/` directory with explicit per-environment files:

```
core/settings/
├─ __init__.py      # empty
├─ env.py           # django-environ bootstrap – single place BASE_DIR and
env() live
├─ base.py          # everything shared across all environments
├─ local.py         # development overrides (DEBUG=True, no HTTPS cookie,
etc.)
├─ production.py    # security hardening (HSTS, SECURE_SSL_REDIRECT, etc.)
└─ test.py          # test runner settings (in-memory Celery, test DB)
```

`env.py` is the single place where `django-environ` is initialised:

```
# core/settings/env.py
import environ
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent.parent
```

```
env = environ.Env()
environ.Env.read_env(BASE_DIR / ".env")
```

Every other settings file imports from it:

```
# core/settings/base.py
from core.settings.env import env, BASE_DIR

SECRET_KEY = env("SECRET_KEY")
DEBUG = env.bool("DEBUG", default=False)
```

The `DJANGO_SETTINGS_MODULE` environment variable selects the active file. Docker Compose sets `core.settings.local`; the test runner uses `core.settings.test` via `pytest.ini`.

Previously `INSTALLED_APPS` was a flat list. It is now split into three named lists to make intent clear:

```
DJANGO_APPS = ["django.contrib.admin", "django.contrib.auth", ...]
THIRD_PARTY_APPS = ["corsheaders", "rest_framework", "drf_spectacular", ...]
LOCAL_APPS = ["accounts", "documents", "qa"]

INSTALLED_APPS = DJANGO_APPS + THIRD_PARTY_APPS + LOCAL_APPS
```

3. Cross-cutting: Serializer Split (Input / Output)

Before: serializers were `ModelSerializer` subclasses used for both reading and writing. Validation logic, uniqueness checks, and `create()` / `update()` ORM operations lived inside them.

After: serializers are split into explicit Input and Output types. Input serializers are plain `serializers.Serializer` subclasses — they validate shape and types only, never touch the ORM. Output serializers are read-only `ModelSerializer` subclasses.

```
# accounts/serializers.py – before
class RegisterSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ("username", "email", "password", "password_confirm")

    def validate(self, attrs):
        if attrs["password"] != attrs["password_confirm"]:
            raise serializers.ValidationError(...)
        if User.objects.filter(email=attrs.get("email")).exists(): # ORM in
serializer
            raise serializers.ValidationError(...)
        return attrs
```

```
def create(self, validated_data): # ORM in serializer
    validated_data.pop("password_confirm")
    user = User.objects.create_user(**validated_data)
    return user
```

```
# accounts/serializers.py – after
class RegisterInputSerializer(serializers.Serializer):
    username = serializers.CharField(max_length=150)
    email = serializers.EmailField()
    password = serializers.CharField(write_only=True, validators=
[validate_password])
    password_confirm = serializers.CharField(write_only=True)

    def validate(self, attrs):
        if attrs["password"] != attrs["password_confirm"]:
            raise serializers.ValidationError({"password": "Passwords do not
match."})
        return attrs
    # No ORM. No create(). Uniqueness is the service's responsibility.

class UserOutputSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ("id", "username", "email", "question_count", "created_at")
        read_only_fields = fields
```

The same pattern is applied across `documents` and `qa`: `DocumentUploadInputSerializer` / `DocumentOutputSerializer`, `AskQuestionInputSerializer` / `AskQuestionOutputSerializer`, and so on.

4. accounts Domain

Services and Selectors

Before: views handled everything directly — ORM calls, uniqueness checks, token blacklisting.

After: `accounts/selectors.py` owns read queries; `accounts/services/user.py` owns mutations.

```
# accounts/selectors.py
def get_user_by_id(*, user_id) -> User:
    try:
        return User.objects.get(pk=user_id)
    except User.DoesNotExist:
        raise UserNotFoundError()

def user_email_exists(*, email: str, exclude_user_id=None) -> bool:
```

```

qs = User.objects.filter(email=email)
if exclude_user_id is not None:
    qs = qs.exclude(pk=exclude_user_id)
return qs.exists()

```

```

# accounts/services/user.py
def register_user(*, username: str, email: str, password: str) -> User:
    if user_email_exists(email=email):
        raise UserAlreadyExistsError(field="email")
    if user_username_exists(username=username):
        raise UserAlreadyExistsError(field="username")

    user = User.objects.create_user(username=username, email=email,
password=password)
    logger.info("New user registered: %s (id=%s)", user.email, user.id)
    return user

def logout_user(*, refresh_token: str | None) -> None:
    if not refresh_token:
        raise MissingRefreshTokenError()
    try:
        token = RefreshToken(refresh_token)
        token.blacklist()
    except TokenError:
        raise InvalidTokenError()

```

Note that `logout_user` catches `TokenError` specifically — the `simplejwt` exception for invalid or expired tokens — rather than bare `Exception`, which would swallow genuine unexpected failures.

Cookie-based Refresh Token

The login view was changed to strip the `refresh` token from the response body and set it as an HTTP-only cookie instead. This removes the refresh token from JavaScript's reach and closes the XSS vector that exists when it lives in `localStorage`.

```

# accounts/views.py – LoginView
def post(self, request, *args, **kwargs):
    response = super().post(request, *args, **kwargs)
    refresh_token = response.data.pop("refresh", None)
    if refresh_token:
        _set_refresh_cookie(response, refresh_token)
    return response

```

A matching `CookieTokenRefreshView` reads the token back from the cookie for the `/refresh/` endpoint, so clients never need to manage the refresh token manually.

View (before and after)

```
# accounts/views.py – RegisterView before
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [permissions.AllowAny]

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        user = serializer.save()
        return Response(UserSerializer(user).data,
                        status=status.HTTP_201_CREATED)
```

```
# accounts/views.py – RegisterView after
class RegisterView(APIView):
    permission_classes = [permissions.AllowAny]

    @extend_schema(
        summary="Register a new user",
        request=RegisterInputSerializer,
        responses={201: UserOutputSerializer},
    )
    def post(self, request: Request) -> Response:
        input_ser = RegisterInputSerializer(data=request.data)
        input_ser.is_valid(raise_exception=True)

        user = register_user(
            username=input_ser.validated_data["username"],
            email=input_ser.validated_data["email"],
            password=input_ser.validated_data["password"],
        )
        return Response(
            {"message": "Account created successfully.", "user":
            UserOutputSerializer(user).data},
            status=status.HTTP_201_CREATED,
        )
```

The view is now a coordinator: validate input with a serializer, call a service, serialize output. It contains no business logic.

5. documents Domain

Thin Task

Before: `documents/tasks.py` was a 95-line Celery task that loaded files, split text, called Ollama, stored embeddings, and managed document status — the entire pipeline inline.

After: the task is a seven-line delegator:

```
# documents/tasks.py – after
from celery import shared_task

@shared_task
def process_document_embedding(doc_id: UUID):
    from documents.services import ProcessDocumentService
    ProcessDocumentService.execute(document_id=doc_id)
```

The embedding pipeline logic moved into `documents/services/embedding.py`, which is independently testable without Celery.

Service Classes

Document operations are encapsulated in service classes inside `documents/services/document.py`:

```
class CreateDocumentService:
    @staticmethod
    def execute(*, user: User, validated_data: dict[str, Any]) -> Document:
        doc = Document.objects.create(user=user, **validated_data)
        logger.info("Document created: id=%s", doc.id)
        transaction.on_commit(
            lambda: cast(Any, current_app).send_task(
                "documents.tasks.process_document_embedding",
                args=[str(doc.id)],
            )
        )
        return doc

class DeleteDocumentService:
    @staticmethod
    def execute(*, user: User, document_id: UUID) -> dict[str, str]:
        doc = document_get(user=user, document_id=document_id)
        data = {"id": str(doc.id), "file_name": doc.file_name}
        doc.delete()
        return data

class GetDocumentStatusService:
    STATUS_DESCRIPTIONS = {
        Document.Status.PENDING: "Waiting to be processed",
        Document.Status.PROCESSING: "Extracting text and generating
embeddings...",
        Document.Status.COMPLETED: "Ready for Q&A",
```

```

        Document.Status.FAILED: "Processing failed. Check file format or
logs.",
    }

    @classmethod
    def execute(cls, *, user: User, document_id: UUID) -> dict[str, str]:
        doc = document_get(user=user, document_id=document_id)
        return {
            "id": str(doc.id),
            "file_name": doc.file_name,
            "status": doc.status,
            "status_description": cls.STATUS_DESCRIPTIONS.get(doc.status,
"Unknown"),
        }

```

`current_app.send_task()` is used instead of importing the task function directly. This eliminates a circular import (`services/document.py` → `tasks.py` → `services/__init__.py` → `services/document.py`) without resorting to a deferred import inside the function body.

6. qa Domain

From Synchronous Blocking to Async Pipeline

Before: `POST /api/qa/ask/` ran the full RAG pipeline inline — embedding, retrieval, reranking, generation — inside a single request/response cycle. Under any real load this caused gunicorn worker timeouts.

After: the endpoint dispatches to a Celery task and returns a 202 immediately. The client polls `GET /api/qa/result/{task_id}/` for the answer.

```

# qa/views.py – before (simplified)
class QuestionAnsweringView(APIView):
    def post(self, request):
        # ... embedding, pgvector query, reranking, ollama call ... all inline
        return Response({"answer": answer, "sources": sources})

```

```

# qa/views.py – after
class QuestionAnsweringView(APIView):
    def post(self, request: Request) -> Response:
        input_ser = AskQuestionInputSerializer(data=request.data)
        input_ser.is_valid(raise_exception=True)

        result = AskQuestionService.execute(
            user=request.user,
            validated_data=input_ser.validated_data,

```



```
)
return Response(result, status=status.HTTP_202_ACCEPTED)
```

```
# qa/services/ask.py
class AskQuestionService:
    @staticmethod
    def execute(*, user: User, validated_data: dict) -> dict:
        from qa.tasks import process_question_task

        doc_id = validated_data.get("document_id")
        if doc_id and not document_exists_for_user(doc_id=doc_id, user=user):
            raise DocumentNotFoundError(...)

        task = cast(Any, process_question_task).delay(
            question=validated_data["question"],
            user_id=str(user.id),
            doc_id=str(doc_id) if doc_id else None,
            page_filter=validated_data.get("page"),
        )
        return {"task_id": str(task.id), "status": "processing"}
```

```
# qa/tasks.py
@shared_task(bind=True)
def process_question_task(self, question, user_id, doc_id, page_filter) -> dict:
    from qa.services.process import ProcessQuestionService
    return ProcessQuestionService.execute(
        question=question,
        user_id=user_id,
        doc_id=doc_id,
        page_filter=page_filter,
        task_id=self.request.id,
    )
```

Services Split

The original `qa/views.py` was 347 lines containing all pipeline logic. The `qa` domain now has seven service modules:

File	Responsibility
<code>services/ask.py</code>	Validate doc ownership, dispatch Celery task
<code>services/process.py</code>	Orchestrate the full RAG pipeline
<code>services/embedding.py</code>	Query embedding with Redis cache (24 h TTL)

File	Responsibility
<code>services/retrieval.py</code>	PGVector similarity search
<code>services/reranking.py</code>	BM25 + LLM hybrid reranking
<code>services/stream.py</code>	Streaming pipeline for NDJSON responses
<code>services/activity.py</code>	QuestionActivity create, list, increment question count

7. Infrastructure

Dockerfile

The base image was pinned from `python:3.12-slim` to `python:3.12-slim-bookworm` for an explicit Debian release. The `collectstatic` call was removed from the image build — it now runs at container start alongside `migrate`, which is correct because static files may depend on runtime settings.

docker-compose.yml

Before: each service (`web`, `celery`, `celery-beat`) repeated the full environment, volume, `depends_on`, and `extra_hosts` configuration.

After: a YAML anchor `x-backend-common` holds the shared definition. Each service merges it with `<<: *backend-common` and only declares what differs:

```
x-backend-common: &backend-common
  build: ./backend
  env_file: .env
  environment:
    DJANGO_SETTINGS_MODULE: core.settings.local
  volumes:
    - ./backend:/app
    - media_volume:/app/media
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
  extra_hosts:
    - "host.docker.internal:host-gateway"
  restart: unless-stopped

services:
  web:
    <<: *backend-common
    command: >
      sh -c "python manage.py migrate &&
        python manage.py collectstatic --noinput &&
        gunicorn core.wsgi:application --bind 0.0.0.0:8000 --workers 2 --
        timeout 30"
```

```
celery:
  <<: *backend-common
  command: celery -A core worker -l info --concurrency=2 -Q
celery,documents,qa
```

Hardcoded credentials (`raguser`, `ragpassword`) were removed from the compose file. The DB healthcheck and container environment now read from `.env` via `${POSTGRES_USER}` / `${POSTGRES_DB}`.

8. API Behaviour Changes

Two changes are breaking from the perspective of an existing API client:

1. Refresh token moved to HTTP-only cookie. Previously the `POST /api/auth/login/` response body contained a `refresh` field. That field is no longer present. The refresh token is now set as an HTTP-only cookie (`refresh_token`, path `/api/auth/`). The `/api/auth/refresh/` and `/api/auth/logout/` endpoints read it from the cookie transparently.

2. Question answering is now asynchronous. Previously `POST /api/qa/ask/` blocked until the answer was ready and returned it directly. It now returns `{"task_id": "...", "status": "processing"}` with a 202. Clients must poll `GET /api/qa/result/{task_id}/` until `status` is no longer `processing`.

9. Test Results

99 passed in 7.94s – 84% overall coverage

Domain	Tests	Notable coverage
accounts	15	views 99%, services 83%, selectors 70%
documents	33	views 100%, services/document.py 92%, services/embedding.py 36%
qa	26	views 100%, services/ask.py 93%, services/reranking.py 24%
integration	4	end-to-end workflow with mocked Ollama and PGVector
workflow	21	full auth + document + QA lifecycle

All Ollama and PGVector calls are mocked. No live services are required to run the suite.

Low coverage in `services/embedding.py` (36%) and `services/reranking.py` (24%) reflects that the Ollama-dependent paths are exercised only through integration-level mocks, not direct unit tests.

10. Known Gaps and Planned Next Steps

Class-based services. The HackSoft styleguide recommends plain functions with keyword-only arguments as services. This codebase uses static-method service classes (`CreateDocumentService.execute(...)`)

throughout. This was a deliberate choice to manage the scope of the current sprint. Refactoring to plain functions is planned for the next sprint and requires no API or test changes — only internal restructuring.

qa/services/reranking.py coverage at 24%. The reranking module is the most complex part of the pipeline and the least covered. Dedicated unit tests with mocked Ollama responses are needed here.

accounts/selectors.py coverage at 70%. `get_all_users` and the `exclude_user_id` branch of `user_username_exists` are not currently exercised by any test.